

PDR — Marketplace Agrícola Huila (MVP)

Fecha: agosto 2026 **Autor:** Elkin Stiven Contreras Rojas, Alejandro Ortiz Vargas

1. Contexto y problema

El sistema busca acercar directamente a productores agrícolas y compradores, reduciendo la dependencia de intermediarios que erosionan el margen del productor. Cada usuario puede tener un perfil dual (comprador y/o vendedor). Los vendedores publican productos; los compradores navegan el catálogo, negocian mediante un canal de mensajería interno y cierran la operación por una de dos vías: pago externo acordado entre las partes, o pago procesado por la plataforma.

1.1 Por qué distribuido y no monolítico

La separación en servicios no es un requisito decorativo de la asignatura: responde a que los subsistemas tienen perfiles de carga, disponibilidad, consistencia y modelo de datos genuinamente distintos.

Subsistema	Perfil de carga	Consistencia	Estado	Modelo de datos
Auth/Usuarios	Baja, picos en login	Fuerte (CP) — no puede haber ambigüedad sobre si alguien está autenticado	Sin estado	Relacional
Catálogo	Muy alta lectura, baja escritura	Eventual (AP) — tolera ver un precio 2 s desactualizado	Sin estado	Relacional
Transacciones	Media, con I/O externo hacia la pasarela	Fuerte (CP) — el ledger no admite saldos ambiguos	Sin estado	Relacional (ACID obligatorio)
Mensajería	Escritura intensiva, conexiones persistentes	Eventual	Con estado (conexiones WebSocket)	Documental, append-only
Notificaciones	Ráfagas, I/O contra SMTP no confiable	Eventual, at-least-once	Sin estado	Ninguno propio

Tres consecuencias concretas de esa tabla:

- Aislamiento de fallos.** Si el proveedor de correo tarda 30 s o la pasarela de pagos se cae, el catálogo y la autenticación no pueden degradarse. Un monolito los acopla en el mismo proceso y en el mismo pool de conexiones.
- Escalado independiente.** El catálogo requiere escalar horizontalmente ante picos de tráfico (cosecha, temporada); Auth no. Escalar un monolito significa replicar también lo que no lo necesita.

3. **Heterogeneidad de almacenamiento.** El ledger exige ACID; el historial de chat es append-only sin joins. Forzar ambos en el mismo motor penaliza a uno de los dos.

1.2 Corrección respecto a la versión 1.0 del PDR

La versión anterior especificaba "*una base de datos SQL (Postgres) compartida*". **Esa decisión queda revocada.** Una base de datos compartida entre servicios constituye el anti-patrón del monolito distribuido: elimina la posibilidad de despliegue, escalado y fallo independientes, y contradice directamente la justificación de la Sección 1.1. Ver Sección 6.

2. Objetivos y alcance

2.1 Objetivo general

Permitir el acercamiento y la transacción directa entre comprador y vendedor de productos agrícolas, reduciendo la necesidad de intermediarios.

2.2 Objetivos específicos

- Permitir registro y autenticación de usuarios con perfil dual (comprador/vendedor).
- Permitir que el vendedor gestione sus productos publicados.
- Permitir que el comprador navegue y filtre el catálogo de productos disponibles.
- Proveer un canal de mensajería interno que permita negociar **sin exponer datos de contacto personales**, con revelación mutua opcional bajo consentimiento explícito de ambas partes.
- Soportar dos vías de cierre de la operación: pago externo (fuera de la plataforma) y pago procesado por la plataforma.
- Registrar el movimiento de fondos en un **ledger de partida doble** auditable e inmutable.
- Permitir calificación mutua tras una transacción efectivamente completada.
- Notificar por correo los eventos relevantes del ciclo de vida.

2.3 Fuera de alcance del MVP

Excluido	Razón
Operación con dinero real	Requiere constitución jurídica y habilitación ante la pasarela. Ver Sección 5.6.
Dispersión efectiva a cuentas bancarias	Se simula mediante ledger. El punto de integración queda documentado.
Logística de transporte y almacenamiento	Fuera de la capacidad de un desarrollador único.
Verificación de identidad con IA (KYC cédula-rostro)	Fase 2. Enfoque ya definido: librerías preentrenadas (face_recognition , DeepFace), sin entrenamiento propio.
Geolocalización con mapa	Solo filtro por municipio/texto.
Moderación automática de contenido	Revisión manual por administrador.

Excluido	Razón
Aplicación móvil nativa	Web responsive.

3. Requisitos

3.1 Funcionales

Autenticación y usuarios

ID	Descripción
RF1	Registro con nombre, correo y contraseña; login con correo/contraseña; emisión de JWT.
RF2	El usuario alterna su rol activo (comprador/vendedor) sobre una misma cuenta.
RF3	El vendedor completa perfil de finca (departamento, municipio, vereda, nombre de finca) como requisito para publicar.
RF4	El perfil público muestra nombre, municipio, promedio de calificación y número de transacciones completadas. No muestra teléfono ni correo.

Catálogo

ID	Descripción
RF5	El vendedor puede crear, editar, despublicar y eliminar productos (nombre, categoría, unidad, cantidad disponible, precio unitario, fotos, municipio, estado).
RF6	El comprador navega el catálogo con filtros por categoría, municipio y rango de precio.
RF7	La ficha de producto muestra el detalle, la reputación del vendedor y las acciones disponibles (iniciar conversación / comprar).

Mensajería

ID	Descripción
RF8	El comprador inicia una conversación desde la ficha del producto. La conversación queda asociada al producto.
RF9	Ambas partes intercambian mensajes de texto en tiempo real. El historial es persistente y consultable.
RF10	Cualquiera de las partes puede solicitar el intercambio de datos de contacto. Los datos se revelan únicamente cuando ambas partes han aceptado.
RF11	Cada mensaje recibido por un usuario desconectado genera una notificación por correo.
RF12	Cualquiera de las partes puede reportar una conversación abusiva o fraudulenta (revisión manual por administrador).

Transacciones y pagos

ID	Descripción
RF13	Desde una conversación, el vendedor genera una oferta formal (producto, cantidad, precio pactado, vía de pago).
RF14	El comprador acepta o rechaza la oferta. La aceptación crea una orden .
RF15	Vía externa: la orden se marca como acordada por fuera. Ambas partes confirman manualmente la culminación. La plataforma no interviene en el dinero.
RF16	Vía plataforma: el comprador paga mediante la pasarela (sandbox). El sistema registra el recaudo en el ledger y retiene el saldo.
RF17	El sistema procesa los webhooks de la pasarela de forma idempotente : un mismo evento recibido N veces produce un único asiento contable.
RF18	El sistema ejecuta una reconciliación periódica contra la API de la pasarela para las órdenes en estado no final, cubriendo el caso de webhook perdido.
RF19	El comprador confirma la recepción del producto, lo que libera el saldo retenido hacia el saldo disponible del vendedor, descontando la comisión de la plataforma.
RF20	Si el comprador no confirma ni disputa en N días (configurable, por defecto 7), el saldo se libera automáticamente.
RF21	El comprador puede abrir una disputa antes del vencimiento, lo que congela el saldo hasta resolución manual del administrador (liberación o reembolso).
RF22	El vendedor consulta su saldo (disponible / retenido) y el extracto de movimientos derivado del ledger.
RF23	El vendedor solicita retiro de su saldo disponible. En el MVP la solicitud queda registrada en estado SIMULADA ; no se ejecuta dispersión real.

Calificaciones

ID	Descripción
RF24	La calificación mutua (1–5 estrellas + comentario) se habilita únicamente sobre órdenes en estado COMPLETADA .
RF25	Las calificaciones son inmutables una vez emitidas y tienen ventana de 30 días.
RF26	El promedio de reputación se propaga al perfil público del usuario mediante evento asíncrono.

Notificaciones

ID	Descripción
RF27	Notificación por correo ante: nuevo mensaje, oferta recibida, oferta aceptada, pago confirmado, entrega pendiente de confirmación, disputa abierta, calificación recibida.

ID	Descripción
RF28	Los envíos fallidos se reintentan con backoff exponencial; tras agotar los reintentos, el mensaje pasa a una cola de mensajes muertos (DLQ) para inspección.

3.2 No funcionales

Categoría	Requisito
Disponibilidad	Catálogo y Auth deben permanecer operativos aunque Mensajería, Transacciones o Notificaciones estén caídos. La caída de Notificaciones no debe perder eventos: quedan encolados y se procesan al recuperarse.
Latencia	Catálogo y Auth < 1 s (p95). Entrega de mensaje de chat < 500 ms entre emisor y receptor conectados.
Consistencia	Fuerte en Auth y en el ledger de Transacciones. Eventual en reputación, catálogo y notificaciones.
Integridad contable	Invariante de suma cero: todo asiento del ledger suma exactamente cero entre sus líneas. Verificable en cualquier momento. Los asientos son inmutables (append-only); una corrección se registra como asiento inverso, nunca como UPDATE ni DELETE .
Idempotencia	Todo consumidor de eventos y todo receptor de webhooks debe ser idempotente. Entrega <i>at-least-once</i> , procesamiento <i>effectively-once</i> .
Seguridad	Contraseñas con bcrypt. Autenticación por JWT validado en el gateway. Los datos de contacto personales nunca se exponen sin consentimiento bilateral registrado (RF10).
Protección de datos	Cumplimiento de la Ley 1581 de 2012 (Habeas Data): consentimiento informado en el registro, finalidad declarada del tratamiento, y no divulgación de datos de contacto por defecto.
Escalabilidad	Catálogo escalable horizontalmente sin estado. Mensajería escalable horizontalmente pese a mantener conexiones con estado (ver Sección 5.2).
Observabilidad	Correlation ID propagado desde el gateway a través de todos los servicios y eventos, para poder trazar una operación completa.

4. Arquitectura

4.1 Componentes

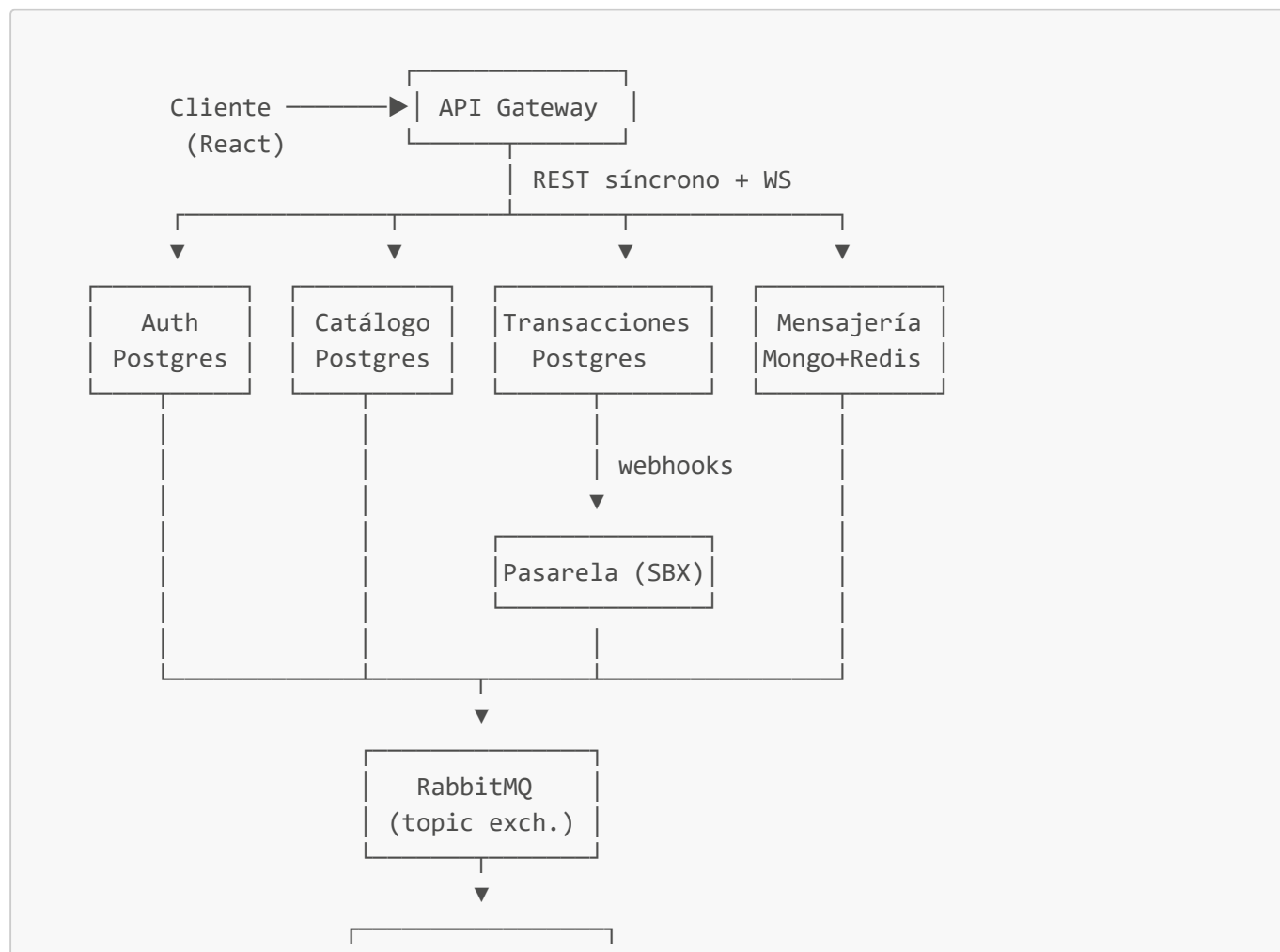
Componente	Tipo	Responsabilidad	Almacén
API Gateway	Enrutador	Punto de entrada único, validación de JWT, rate limiting, correlation ID	—
Auth/Usuarios	Servicio REST	Registro, login, JWT, perfil dual, perfil de finca, reputación desnormalizada	PostgreSQL
Catálogo	Servicio REST	Productos, categorías, filtros, búsqueda	PostgreSQL

Componente	Tipo	Responsabilidad	Almacén
Transacciones	Servicio REST + orquestador	Ofertas, órdenes, máquina de estados, integración con pasarela, ledger , disputas, calificaciones	PostgreSQL
Mensajería	Servicio WebSocket + REST	Conversaciones, mensajes, consentimiento de contacto	MongoDB + Redis
Notificaciones	Worker consumidor	Consumo de eventos, plantillas, envío SMTP, reintentos, DLQ	—

Nota sobre Notificaciones: no es un servicio REST. No expone API y nadie lo invoca; únicamente consume de RabbitMQ. Su justificación no es el tamaño de su dominio (que es mínimo) sino el **aislamiento de fallos**: es el único componente que hace I/O contra un sistema externo poco confiable (SMTP), y esa latencia y esos reintentos no deben vivir dentro de ningún servicio de dominio.

Nota sobre Calificaciones: en la versión 1.0 figuraba como servicio independiente. **Queda fusionado dentro de Transacciones.** Justificación: (a) su dominio es un CRUD anémico; (b) no tiene perfil propio de carga, disponibilidad ni consistencia, que es el criterio de separación adoptado en la Sección 1.1; (c) una calificación no puede existir sin una orden completada, de modo que separarlos obligaría a un acoplamiento síncrono permanente entre ambos servicios sin ganancia alguna. Su ubicación natural es como estado final de la máquina de estados de la orden.

4.2 Comunicación



Notificaciones
(worker)

Síncrono (REST): cliente → gateway → servicios. Consultas que requieren respuesta inmediata.

Asíncrono (RabbitMQ, exchange tipo *topic*): todo lo que puede tolerar retraso o cuyo fallo no debe propagarse.

Evento	Emisor	Consumidores
<code>chat.mensaje.creado</code>	Mensajería	Notificaciones
<code>chat.contacto.revelado</code>	Mensajería	Notificaciones
<code>orden.creada</code>	Transacciones	Notificaciones
<code>orden.pagada</code>	Transacciones	Notificaciones, Catálogo (descuenta disponibilidad)
<code>orden.completada</code>	Transacciones	Notificaciones
<code>orden.disputada</code>	Transacciones	Notificaciones
<code>calificacion.creada</code>	Transacciones	Notificaciones, Auth/Usuarios (recalcula promedio)
<code>usuario.perfil.actualizado</code>	Auth/Usuarios	Catálogo (actualiza datos del vendedor en las fichas)

4.3 Consistencia eventual: dónde y por qué

El caso demostrativo es la **reputación**. Cuando se emite una calificación, Transacciones publica `calificacion.creada`; Auth/Usuarios lo consume y recalcula el promedio almacenado en el perfil. Entre ambos momentos existe una ventana de inconsistencia de milisegundos a segundos.

Esa ventana es aceptable —a nadie le afecta ver un promedio de 4.2 en lugar de 4.3 durante un segundo— y a cambio se evita que Catálogo y Auth tengan que llamar a Transacciones en cada renderizado de una ficha de producto, lo que acoplaría su disponibilidad a la de un tercer servicio.

Contraejemplo deliberado: el saldo del ledger **no** es eventualmente consistente. Vive completo dentro de una transacción ACID de PostgreSQL en el servicio de Transacciones. Ningún otro servicio puede escribirlo. Esta asimetría —eventual donde es barata, fuerte donde es obligatoria— es la decisión arquitectónica central del sistema.

4.4 Patrón Outbox

Existe un problema de doble escritura: si Transacciones escribe en PostgreSQL y luego publica en RabbitMQ, un fallo entre ambas operaciones deja el sistema inconsistente (orden pagada sin evento emitido).

Solución: patrón *outbox*. El evento se inserta en una tabla `outbox` dentro de la misma transacción ACID que la escritura de dominio. Un proceso aparte lee esa tabla y publica a RabbitMQ, marcando lo publicado. Si el publicador falla, reintenta; el peor caso es una publicación duplicada, que los consumidores idempotentes absorben.

5. Decisiones de diseño clave

5.1 Mensajería: doble propósito

El chat interno reemplaza la exposición directa de teléfono y correo en la ficha de producto (diseño de la versión 1.0). Cumple dos funciones:

1. **Reduce el pasivo de protección de datos.** Publicar el teléfono personal de un productor en una página pública es un tratamiento de datos sin finalidad acotada bajo la Ley 1581 de 2012, además de una vía directa de spam y fraude.
2. **Provee evidencia verificable de contacto.** La versión 1.0 habilitaba calificaciones mediante confirmación mutua, esquema que dos cuentas cómplices pueden falsear trivialmente. Con mensajería, la existencia de una conversación con mensajes de ambas partes es un requisito previo comprobable.

Riesgo asumido: el usuario rural prefiere WhatsApp. Un chat que nadie revisa mata negociaciones.

Mitigación: (a) toda recepción de mensaje con el destinatario desconectado dispara correo; (b) el flujo de revelación bilateral de contacto (RF10) permite migrar la conversación a WhatsApp de forma explícita y consentida, conservando el registro de que hubo contacto. El chat no compite con WhatsApp: lo antecede y lo autoriza.

5.2 Mensajería: WebSocket + Redis Pub/Sub

Transporte: WebSocket con STOMP sobre Spring Boot.

El problema de escalar un servicio con estado. Con múltiples instancias del servicio de Mensajería tras un balanceador, el usuario A puede estar conectado a la instancia 1 y el usuario B a la instancia 2. Un mensaje de A hacia B se procesa en la instancia 1, que no tiene la conexión de B. El mensaje se pierde.

Solución: Redis Pub/Sub como relay entre instancias. Cada instancia se suscribe a los canales de los usuarios que tiene conectados. Al recibir un mensaje, la instancia lo persiste en MongoDB y lo publica al canal Redis del destinatario; la instancia que mantenga esa conexión lo recibe y lo entrega por WebSocket.

Este es el punto técnicamente más difícil del proyecto y también el de mayor valor demostrativo para la asignatura: es la respuesta concreta a "*¿cómo se escala horizontalmente un componente con estado?*".

Persistencia en MongoDB: justificada por el perfil de la carga —append-only, sin joins, particionable por conversación, alto volumen de escritura, sin necesidad de consistencia fuerte—. No es una elección estética; es el único punto del sistema donde el modelo documental gana al relacional.

Plan de contingencia: si en la semana 10 del cronograma el WebSocket no está estable, se degrada a polling HTTP cada 5 s sobre el mismo modelo de datos, y la evolución a WebSocket se documenta como trabajo futuro. El ledger no se recorta bajo ninguna circunstancia; es la pieza de mayor originalidad del proyecto.

5.3 Pagos: modelo híbrido recaudo-pasarela / dispersión-ledger

El sistema ofrece dos vías de cierre:

Vía A — externa. Las partes acuerdan pago fuera de la plataforma (efectivo, transferencia, Nequi directo). El sistema solo registra la orden y su culminación declarada. No toca dinero. Es la vía realista para el usuario rural sin bancarización.

Vía B — plataforma. El comprador paga por la pasarela; el sistema retiene el saldo hasta la confirmación de entrega y luego lo libera al vendedor descontando comisión.

En la Vía B, el **recaudo** se integra contra el sandbox de la pasarela (transacciones reales de API, dinero ficticio), mientras que la **dispersión** se resuelve internamente con un ledger de partida doble en lugar de invocar la API de pagos a terceros. Esto es deliberado:

- El recaudo es donde vive la complejidad distribuida interesante: webhooks, idempotencia, reconciliación, estados intermedios, compensación.
- La dispersión real no aporta complejidad arquitectónica nueva (es una llamada HTTP más), pero sí exige habilitación legal y cuentas bancarias verificadas de los vendedores.
- El ledger aporta más valor que la dispersión real: es lógica de dominio propia, no integración de terceros. Para efectos del registro DNDA, es código original.

5.4 El ledger de partida doble

Principio: el saldo de un usuario no es un campo que se actualiza. Es la **suma de las líneas** de todos los asientos que lo afectan. Nunca se ejecuta `UPDATE saldo = saldo + x`.

Cuentas contables:

Cuenta	Naturaleza
<code>plataforma:recaudo</code>	Fondos ingresados por la pasarela
<code>usuario:{id}:retenido</code>	Saldo del vendedor pendiente de confirmación de entrega
<code>usuario:{id}:disponible</code>	Saldo del vendedor liberado, retirable
<code>plataforma:comisiones</code>	Ingreso de la plataforma
<code>plataforma:reembolsos</code>	Salida por disputas resueltas a favor del comprador
<code>plataforma:retiros_pendientes</code>	Saldo comprometido por solicitudes de retiro no ejecutadas (SIMULADA en el MVP)

Convención de signos: `plataforma:recaudo` es la cuenta de contrapartida frente al exterior (la pasarela). Al ingresar dinero se acredita en negativo y se abona en positivo a la cuenta del vendedor, de modo que el asiento suma cero sin necesidad de una cuenta externa ficticia. En régimen permanente el saldo acumulado de `plataforma:recaudo` es negativo y su valor absoluto equivale al total de fondos que la plataforma custodia por cuenta de terceros: es un invariante verificable adicional.

Estructura: un asiento (`asiento_contable`) agrupa N líneas (`linea_asiento`), cada una con cuenta, monto con signo y referencia. **Invariante:** la suma de los montos de las líneas de un asiento es exactamente cero. Se verifica en la aplicación y se refuerza con una restricción en base de datos. Un test de integridad recorre todo el ledger y comprueba que el total global es cero.

Flujo de asientos:

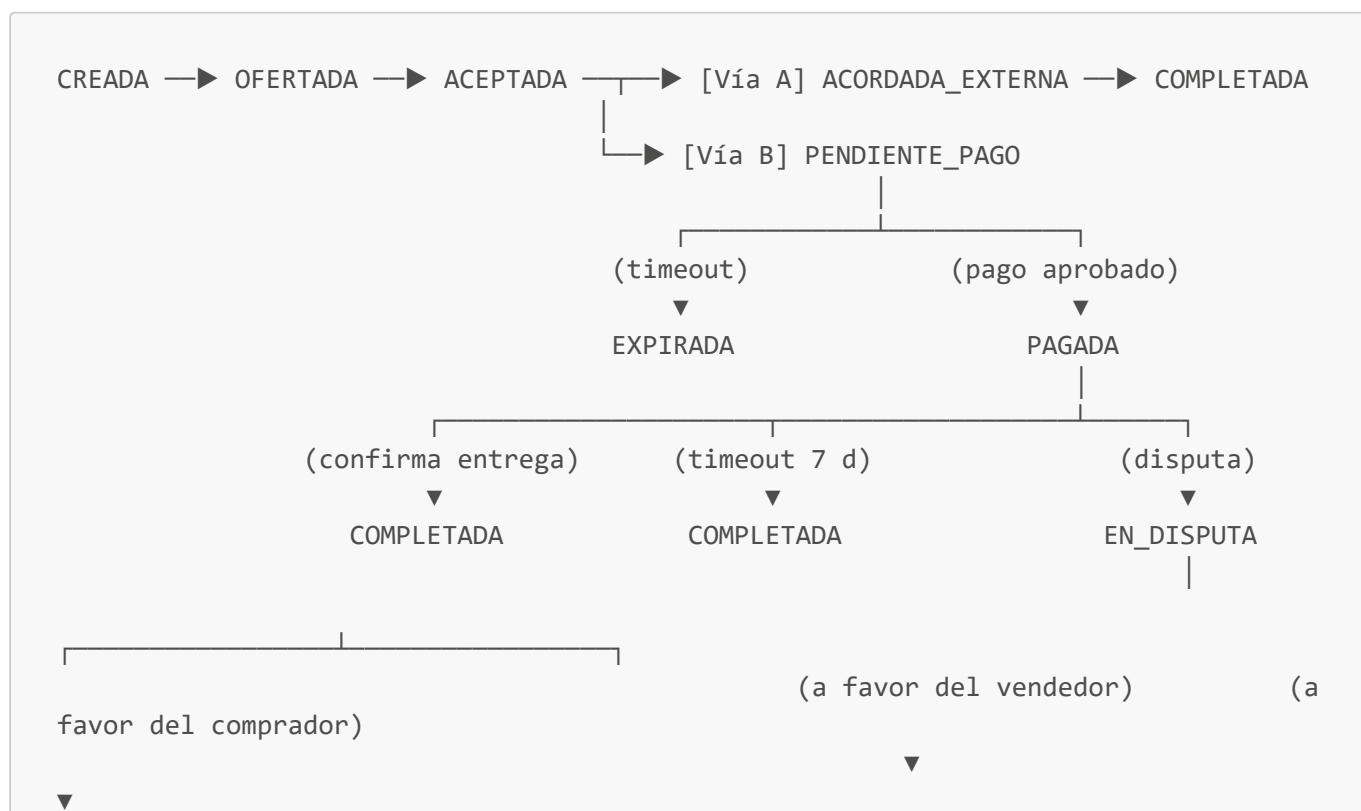
Momento	Líneas del asiento	Σ
Pago aprobado (webhook)	-monto en plataforma:recaudo +monto en usuario:{vendedor}:retenido	0
Entrega confirmada	-monto en usuario:{vendedor}:retenido +(monto - comisión) en usuario: {vendedor}:disponible +comisión en plataforma:comisiones	0
Disputa a favor del comprador	-monto en usuario:{vendedor}:retenido +monto en plataforma:reembolsos	0
Retiro solicitado	-monto en usuario:{vendedor}:disponible +monto en plataforma:retiros_pendientes	0
Compensación por webhook tardío (orden EXPIRADA)	-monto en usuario:{vendedor}:retenido +monto en plataforma:reembolsos	0

Ninguna de estas operaciones ejecuta **UPDATE** sobre un saldo. Toda corrección posterior se registra como un asiento nuevo de signo inverso.

Idempotencia: tabla **evento_pasarela_procesado** con restricción **UNIQUE** sobre el identificador de transacción de la pasarela. El webhook consulta esa tabla dentro de la misma transacción ACID que crea el asiento. Un evento repetido choca contra la restricción y se descarta sin efecto.

Reconciliación: job programado que consulta el estado de las órdenes no finales contra la API de la pasarela, cubriendo el caso de webhook nunca entregado. La documentación de Wompi recomienda explícitamente esta verificación periódica del estado de la transacción hasta alcanzar un estado final.

5.5 Máquina de estados de la orden



REEMBOLSADA

COMPLETADA

Solo desde **COMPLETADA** se habilita la calificación mutua. La transición a **REEMBOLSADA** y la resolución de **EN_DISPUTA** requieren intervención de administrador.

Compensación: si un pago se aprueba después de que la orden expiró (webhook tardío sobre orden **EXPIRADA**), la saga ejecuta un asiento de reembolso automático. Es el caso de compensación explícito del sistema.

5.6 Advertencia regulatoria

Retener fondos de terceros no es una decisión meramente técnica. El Banco de la República ha señalado que la viabilidad de los esquemas de pasarela con función de recaudo (modelo agregador) está sujeta a autorizaciones de la Superintendencia Financiera, dado que puede tipificar captación de recursos del público o actividades propias de administradores de sistemas de pago de bajo valor.

Por esta razón el MVP opera exclusivamente en sandbox. El paso a dinero real queda condicionado a: constitución de persona jurídica, asesoría legal sobre el modelo de agregación, y habilitación comercial ante la pasarela. Esta restricción se declara explícitamente para evitar que el proyecto académico se presente como operable en producción.

6. Modelo de datos y persistencia

Base de datos por servicio. Ningún servicio accede al almacén de otro. Las relaciones entre servicios se resuelven por identificador y por eventos, nunca por join.

Implementación práctica para un desarrollador único: una única instancia de PostgreSQL en Docker Compose, con **un esquema por servicio y un usuario de base de datos distinto por servicio**, sin permisos cruzados. No hay claves foráneas entre esquemas. La separación es lógica y está reforzada por permisos; en producción serían instancias independientes. Esta decisión se documenta como tal para no presentarla como aislamiento físico.

Desnormalización deliberada:

Dato duplicado	Dueño	Copia en	Motivo
Nombre y municipio del vendedor	Auth/Usuarios	Catálogo	Evitar llamada síncrona al renderizar cada ficha
Promedio de reputación	Auth/Usuarios (calculado desde eventos de Transacciones)	Catálogo	Ídem
Snapshot de producto (nombre, precio, unidad al momento de la orden)	Catálogo	Transacciones	Inmutabilidad: si el vendedor cambia el precio después, la orden debe conservar el precio pactado

El tercer caso no es una optimización sino un requisito de corrección: una orden histórica no puede depender del estado actual del catálogo.

7. Stack tecnológico

Capa	Tecnología	Justificación
Frontend	React	Se descarta Angular. Mantener dos frameworks no aporta nada y duplica el esfuerzo.
Gateway	Spring Cloud Gateway	Coherente con el resto del backend.
Auth/Usuarios	Spring Boot + PostgreSQL	Spring Security con JWT resuelve el caso de forma madura.
Catálogo	Spring Boot + PostgreSQL	—
Transacciones	Spring Boot + PostgreSQL	El ledger exige transacciones ACID; JPA + @Transactional es adecuado.
Mensajería	Spring Boot (WebSocket/STOMP) + MongoDB + Redis	Ver 5.2.
Notificaciones	Spring Boot	Consumidor de RabbitMQ. Python es viable dado el perfil del desarrollador y mantiene el argumento políglota con bajo riesgo.
Mensajería asíncrona	RabbitMQ	Exchange tipo topic, colas durables, DLQ.
Pasarela	Wompi (sandbox)	Documentación en español, ambiente de pruebas, soporte de idempotencia y webhooks.
Orquestación local	Docker Compose	—

Se descarta Go. Figuraba en la versión 1.0 sin justificación. Introducir un lenguaje nuevo mientras se construyen cuatro servicios en solitario es un riesgo de cronograma sin beneficio proporcional. El argumento políglota queda cubierto por el worker en Python.